

CV-20: Camera QoS & Message Synchronisation

Technical Investigation and Validation Report

Field	Details
Student Name	Sarvesh Venkatesh Vidya
Student ID	S225162255
Unit	SIT782 – Capstone Team Project (B)
Project	Redback Operations – E-Waste Robotic Battery Extraction
Sub-team	Computer Vision
Contribution	CV-20 – Camera QoS & Message Synchronisation

Runtime evidence from the ROS 2 Jazzy / Gazebo perception pipeline

1. Executive Summary

This report documents my work on CV-20, Camera QoS & Message Synchronisation, within the Computer Vision sub-team of the Redback Operations E-Waste Robotic Battery Extraction project. The purpose of the task was to validate how the Gazebo overhead camera communicates with the ROS 2 perception pipeline, with attention to Quality of Service (QoS), message timing, frame identifiers, publishing rates and integration with the phone/battery detector.

The investigation was performed against the running system rather than assuming that the existing settings were correct. Runtime inspection showed that `/top_camera/image` and `/top_camera/camera_info` are published by `top_camera_bridge` using RELIABLE reliability and VOLATILE durability. The `onnx_phone_detector` subscribes to both streams with the same reliability and durability settings, so no QoS compatibility mismatch was observed during testing.

The image and CameraInfo streams use the same frame ID, `overhead_camera/mount/overhead_camera`. Their observed rates were approximately 0.79 Hz and 0.76 Hz respectively. Timestamp sampling showed both streams progressing in 0.5-second simulation-time increments. The custom `/phone_detections` interface was also verified and was observed at approximately 0.80 Hz. Importantly, the sampled image and CameraInfo timestamps were collected in separate time windows, so this report does not claim exact message-to-message synchronisation.

2. Project Context and Contribution

The wider project aims to support robotic extraction of batteries from electronic waste. The Computer Vision sub-team provides the perception layer that receives images from the simulated overhead camera, performs detection and segmentation, and publishes structured detection information for downstream robotic components.

CV-20 sits at the ROS 2 integration boundary. Camera QoS is not merely a configuration detail: it affects whether perception nodes receive camera data and whether camera metadata can be used consistently. I therefore approached the task as a runtime validation exercise: inspect the actual endpoints, measure stream behaviour, compare timing information, and document the limits of what the evidence supports.

Item	Validated configuration / observation
Image topic	<code>/top_camera/image</code>
Camera calibration topic	<code>/top_camera/camera_info</code>
Image publisher	<code>top_camera_bridge</code>
Perception subscriber	<code>onnx_phone_detector</code>
Image format	<code>sensor_msgs/msg/Image</code> , RGB8
CameraInfo format	<code>sensor_msgs/msg/CameraInfo</code>
Image resolution	1280 × 960
Common frame ID	<code>overhead_camera/mount/overhead_camera</code>
Detection output	<code>/phone_detections</code>

3. Objectives

- Inspect the ROS 2 QoS profiles used by the overhead image and CameraInfo streams.
- Verify that the perception node subscriptions are compatible with the camera publisher QoS.
- Inspect message timestamps and frame IDs to assess temporal and camera-frame consistency.
- Measure observed publishing rates for image and CameraInfo messages in the Gazebo/WSL runtime.
- Verify the integrated perception node and custom PhoneDetectionArray interface.
- Record limitations explicitly rather than treating similar rates or frame IDs as proof of exact synchronisation.

4. Test Environment and Method

Testing was carried out in the project's Ubuntu 24.04 WSL2 environment using ROS 2 Jazzy and the team7 perception workspace. The Gazebo simulation and perception node were started using the project's existing scripts. No QoS configuration was changed during this investigation. This was intentional: the existing runtime behaviour was validated first so that any later configuration change would be evidence-driven.

Key ROS 2 inspection commands used were:

```
ros2 topic info /top_camera/image --verbose
ros2 topic info /top_camera/camera_info --verbose
ros2 topic echo /top_camera/image --once
ros2 topic echo /top_camera/camera_info --once
ros2 topic hz /top_camera/image
ros2 topic hz /top_camera/camera_info
ros2 topic echo /top_camera/image --field header --qos-reliability reliable
ros2 topic echo /top_camera/camera_info --field header --qos-reliability reliable
ros2 node list
ros2 topic info /phone_detections --verbose
ros2 interface show team7_msgs/msg/PhoneDetectionArray
ros2 topic hz /phone_detections
```

5. QoS Investigation

5.1 Overhead Image Topic

The /top_camera/image topic is published by top_camera_bridge as sensor_msgs/msg/Image. The publisher reports RELIABLE reliability and VOLATILE durability. The onnx_phone_detector subscription reports the same settings. The separate camera_view subscriber uses BEST_EFFORT, but that is not the detector subscription and therefore was treated separately.

```

Publisher count: 1
Node name: top_camera_bridge
Node namespace: /
Topic type: sensor_msgs/msg/Image
Topic type hash: RIHSB1_d31d41a9a4c4bc8eae9be757b0beed386564f7526c88ea6a4588fb9582527d47
Endpoint type: PUBLISHER
GID: 01.0f.92.9d.25.04.1f.4c.00.00.00.00.00.15.03
QoS profile:
Reliability: RELIABLE
History (Depth): UNKNOWN
Durability: VOLATILE
Lifespan: Infinite
Deadline: Infinite
Liveliness: AUTOMATIC
Liveliness lease duration: Infinite

Subscription count: 2
Node name: camera_view
Node namespace: /
Topic type: sensor_msgs/msg/Image
Topic type hash: RIHSB1_d31d41a9a4c4bc8eae9be757b0beed386564f7526c88ea6a4588fb9582527d47
Endpoint type: SUBSCRIPTION
GID: 01.0f.92.9d.22.05.8c.c4.00.00.00.00.00.13.04
QoS profile:
Reliability: BEST_EFFORT
History (Depth): UNKNOWN
Durability: VOLATILE
Lifespan: Infinite
Deadline: Infinite
Liveliness: AUTOMATIC
Liveliness lease duration: Infinite

Node name: onnx_phone_detector
Node namespace: /
Topic type: sensor_msgs/msg/Image
Topic type hash: RIHSB1_d31d41a9a4c4bc8eae9be757b0beed386564f7526c88ea6a4588fb9582527d47
Endpoint type: SUBSCRIPTION
GID: 01.0f.92.9d.90.05.38.eb.00.00.00.00.00.15.04
QoS profile:
Reliability: RELIABLE
History (Depth): UNKNOWN
Durability: VOLATILE
Lifespan: Infinite
Deadline: Infinite
Liveliness: AUTOMATIC
Liveliness lease duration: Infinite

sarv@LAPTOP-RL0935SC:~$
```

Figure 1. Runtime QoS inspection for /top_camera/image showing the top camera publisher and subscribers.

5.2 CameraInfo Topic

The /top_camera/camera_info topic is also published by top_camera_bridge with RELIABLE reliability and VOLATILE durability. The onnx_phone_detector subscription uses the same settings. The runtime evidence therefore shows compatible QoS between each camera publisher and the perception subscriber.

```

sarv@LAPTOP-RL0935SC:~$ ros2 topic info /top_camera/camera_info --verbose
Type: sensor_msgs/msg/CameraInfo
Publisher count: 1
Node name: top_camera_bridge
Node namespace: /
Topic type: sensor_msgs/msg/CameraInfo
Topic type hash: RIHSB1_b3df68ff46c9d56c88fd3bd4ed22c7a4ddce8c8348f2f59c299e73118e7e275
Endpoint type: PUBLISHER
GID: 01.0f.92.9d.25.04.1f.4c.00.00.00.00.00.16.03
QoS profile:
Reliability: RELIABLE
History (Depth): UNKNOWN
Durability: VOLATILE
Lifespan: Infinite
Deadline: Infinite
Liveliness: AUTOMATIC
Liveliness lease duration: Infinite

Subscription count: 2
Node name: camera_view
Node namespace: /
Topic type: sensor_msgs/msg/CameraInfo
Topic type hash: RIHSB1_b3df68ff46c9d56c88fd3bd4ed22c7a4ddce8c8348f2f59c299e73118e7e275
Endpoint type: SUBSCRIPTION
GID: 01.0f.92.9d.22.05.8c.c4.00.00.00.00.00.14.04
QoS profile:
Reliability: BEST_EFFORT
History (Depth): UNKNOWN
Durability: VOLATILE
Lifespan: Infinite
Deadline: Infinite
Liveliness: AUTOMATIC
Liveliness lease duration: Infinite

Node name: onnx_phone_detector
Node namespace: /
Topic type: sensor_msgs/msg/CameraInfo
Topic type hash: RIHSB1_b3df68ff46c9d56c88fd3bd4ed22c7a4ddce8c8348f2f59c299e73118e7e275
Endpoint type: SUBSCRIPTION
GID: 01.0f.92.9d.90.05.38.eb.00.00.00.00.00.14.04
QoS profile:
Reliability: RELIABLE
History (Depth): UNKNOWN
Durability: VOLATILE
Lifespan: Infinite
Deadline: Infinite
Liveliness: AUTOMATIC
```

Figure 2. Runtime QoS inspection for /top_camera/camera_info showing the top camera publisher and detector subscriber.

5.3 QoS Summary

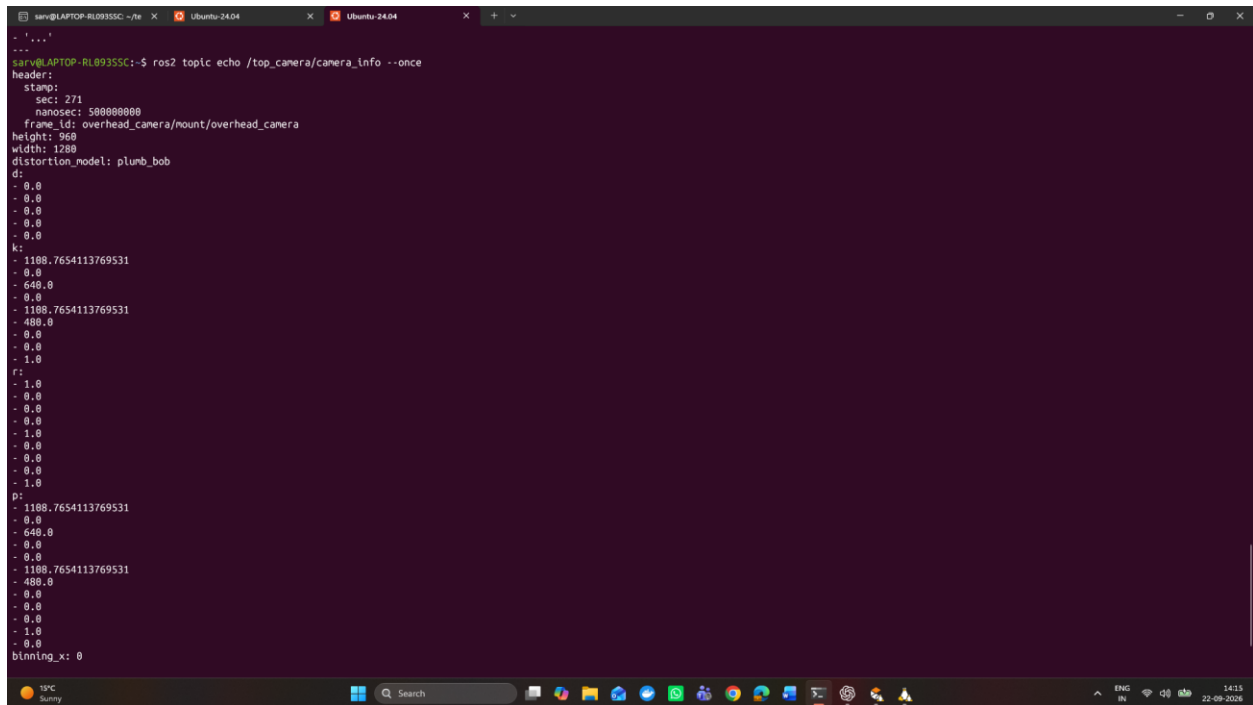
Topic	Publisher	Detector subscriber	Result
/top_camera/image	RELIABLE / VOLATILE	RELIABLE / VOLATILE	Compatible; no mismatch observed
/top_camera/camera_info	RELIABLE / VOLATILE	RELIABLE / VOLATILE	Compatible; no mismatch observed

The engineering outcome is therefore a validated existing configuration rather than a forced QoS change. Since no incompatibility was observed, changing the profile solely to create a code modification would add risk without evidence of a problem.

6. Camera Message and Calibration Parameters

A CameraInfo message was sampled from the running system. It reported a 1280×960 image size and the frame ID overhead_camera/mount/overhead_camera. The intrinsic matrix contained focal values of approximately 1108.7654 pixels in x and y, with the principal point at (640, 480). The sampled distortion coefficients were all zero and the distortion model was plumb_bob.

These parameters matter to the perception pipeline because image coordinates are later used for geometric reasoning. The matching frame identifier on image and CameraInfo messages is also a useful consistency check: both streams identify the same simulated overhead camera frame.



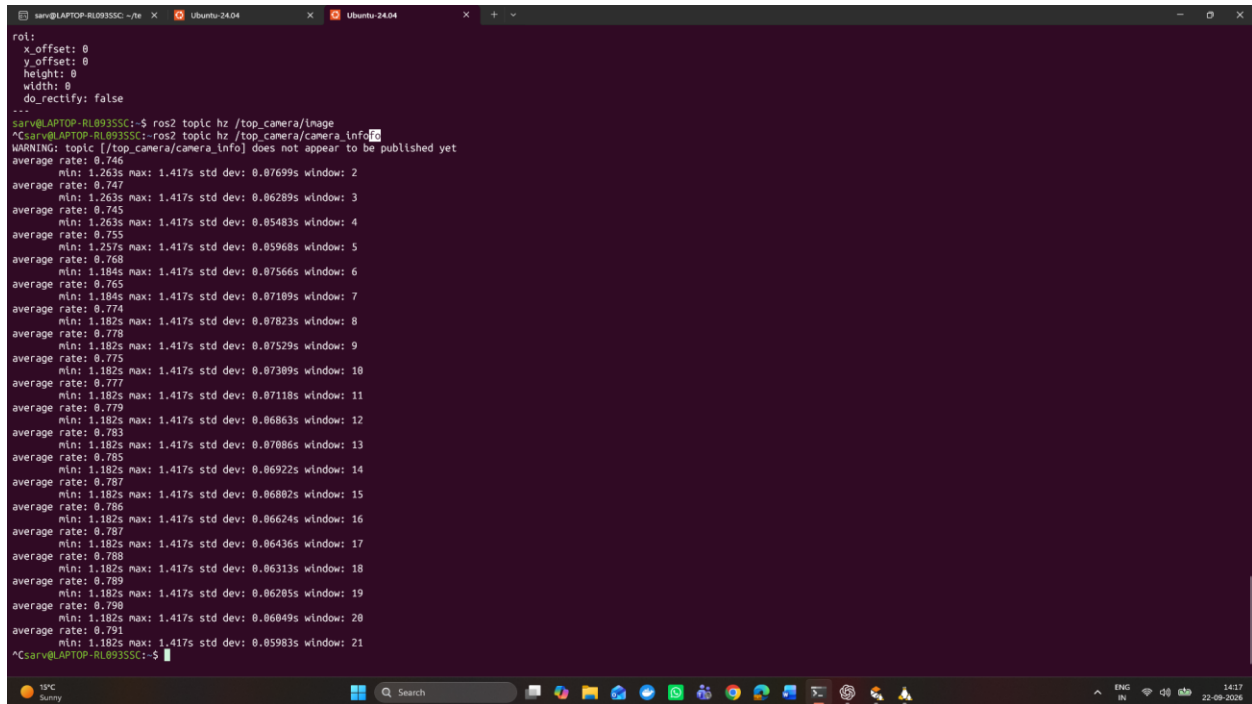
```
...
...
sarv@LAPTOP-RL0935SC:~$ ros2 topic echo /top_camera/camera_info --once
header:
  stamp:
    sec: 271
    nanosec: 588888888
  frame_id: overhead_camera/mount/overhead_camera
height: 960
width: 1280
distortion_model: plumb_bob
d:
- 0.0
- 0.0
- 0.0
- 0.0
- 0.0
- 0.0
k:
- 1108.7654113769531
- 0.0
- 640.0
- 0.0
- 1108.7654113769531
- 480.0
- 0.0
- 0.0
- 1.0
r:
- 1.0
- 0.0
- 0.0
- 0.0
- 1.0
- 0.0
- 0.0
- 0.0
- 1.0
p:
- 1108.7654113769531
- 0.0
- 640.0
- 0.0
- 0.0
- 1108.7654113769531
- 480.0
- 0.0
- 0.0
- 1.0
- 0.0
- 0.0
binning_x: 0
```

Figure 3. Sample /top_camera/camera_info message showing timestamp, frame ID, resolution and intrinsic parameters.

7. Publishing Rate and Timing Analysis

7.1 Image Stream

The /top_camera/image stream was measured using ros2 topic hz. The observed average rate stabilised at approximately 0.79 Hz. The measured inter-message interval ranged from about 1.182 seconds to 1.417 seconds, with the final reported standard deviation around 0.060 seconds.



```
ros2
x_offset: 0
y_offset: 0
height: 0
width: 0
do_rectify: false
...
sarv@LAPTOP-RL8935SC:~$ ros2 topic hz /top_camera/image
^Csarv@LAPTOP-RL8935SC:~$ ros2 topic hz /top_camera/image
WARNING: topic [/top_camera/camera_info] does not appear to be published yet
average rate: 0.746
min: 1.263s max: 1.417s std dev: 0.07699s window: 2
average rate: 0.747
min: 1.263s max: 1.417s std dev: 0.06289s window: 3
average rate: 0.745
min: 1.263s max: 1.417s std dev: 0.05483s window: 4
average rate: 0.755
min: 1.257s max: 1.417s std dev: 0.05968s window: 5
average rate: 0.768
min: 1.184s max: 1.417s std dev: 0.07566s window: 6
average rate: 0.765
min: 1.184s max: 1.417s std dev: 0.07109s window: 7
average rate: 0.774
min: 1.182s max: 1.417s std dev: 0.07823s window: 8
average rate: 0.778
min: 1.182s max: 1.417s std dev: 0.07529s window: 9
average rate: 0.775
min: 1.182s max: 1.417s std dev: 0.07309s window: 10
average rate: 0.777
min: 1.182s max: 1.417s std dev: 0.07118s window: 11
average rate: 0.779
min: 1.182s max: 1.417s std dev: 0.06863s window: 12
average rate: 0.783
min: 1.182s max: 1.417s std dev: 0.07086s window: 13
average rate: 0.785
min: 1.182s max: 1.417s std dev: 0.06922s window: 14
average rate: 0.787
min: 1.182s max: 1.417s std dev: 0.06882s window: 15
average rate: 0.786
min: 1.182s max: 1.417s std dev: 0.06624s window: 16
average rate: 0.787
min: 1.182s max: 1.417s std dev: 0.06436s window: 17
average rate: 0.788
min: 1.182s max: 1.417s std dev: 0.06313s window: 18
average rate: 0.789
min: 1.182s max: 1.417s std dev: 0.06205s window: 19
average rate: 0.790
min: 1.182s max: 1.417s std dev: 0.06049s window: 20
average rate: 0.791
min: 1.182s max: 1.417s std dev: 0.05983s window: 21
^Csarv@LAPTOP-RL8935SC:~$
```

Figure 4. Runtime rate measurement for /top_camera/image; the measured average converges to approximately 0.79 Hz.

7.2 CameraInfo Stream

The /top_camera/camera_info stream was measured independently. Its observed average rate stabilised at approximately 0.76 Hz. The measured interval ranged from about 1.186 seconds to 1.483 seconds, with a final standard deviation of approximately 0.086 seconds.

Stream	Observed average	Observed interval	Interpretation
/top_camera/image	~0.79 Hz	1.182–1.417 s	Slow but reasonably regular in the tested runtime
/top_camera/camera_info	~0.76 Hz	1.186–1.483 s	Similar behaviour to image stream

The two streams therefore behaved at similar observed rates. These values should not be presented as the camera’s nominal configured rate; they are measurements from the active Gazebo/WSL environment. No controlled QoS comparison was performed, so the low rate was not attributed to QoS.

8. Timestamp and Synchronisation Analysis

To avoid printing the full image payload, the header fields of successive messages were inspected. The image stream produced timestamps progressing as 488.5, 489.0, 489.5, 490.0, 490.5, and so on. The CameraInfo stream similarly progressed as 500.5, 501.0, 501.5, 502.0, 502.5, and so on.

Within each stream, the timestamps progressed consistently in 0.5-second simulation-time increments. Both streams also used the same frame ID. These observations support temporal consistency at the stream level.

However, the two sequences were captured in separate sampling windows. They therefore do not contain overlapping timestamps and cannot establish that an individual image and its corresponding CameraInfo message were synchronised at message level. This distinction is important: similar publishing rates and a common frame ID are useful indicators, but they are not proof of exact pairing.

Evidence	Observation	Conclusion
Frame ID	Same frame ID on image and CameraInfo	Camera-frame identity is consistent
Within-stream timestamps	Both streams progress in 0.5 s increments in sampled data	Temporal progression is consistent
Publishing rate	Image ~0.79 Hz; CameraInfo ~0.76 Hz	Streams operate at similar rates
Overlapping timestamp pair	Not captured in the same sampling window	Exact pairwise synchronisation not demonstrated

9. Perception Pipeline Integration Check

The running ROS 2 graph was inspected and included both top_camera_bridge and onnx_phone_detector. The /phone_detections topic was confirmed as a team7_msgs/msg/PhoneDetectionArray publisher owned by onnx_phone_detector.

The custom message interface was successfully loaded with ros2 interface show. It contains the fields used by the team's detection pipeline, including bounding-box coordinates, four ordered bounding-box corners, segmentation centroid, minimum-area-rectangle information, angle and optional world coordinates.

The /phone_detections stream was subsequently measured at approximately 0.80 Hz. Its observed interval varied more than the camera streams, including a maximum observed interval of approximately 3.010 seconds. This is an integration observation only; the detector's processing time can affect its output rate, so it cannot by itself be interpreted as a camera-drop measurement.

10. Overall Results

Validation area	Result	Status
Image QoS	RELIABLE + VOLATILE publisher and detector subscriber	Validated
CameraInfo QoS	RELIABLE + VOLATILE publisher and detector subscriber	Validated
Image resolution	1280 × 960	Validated
Common frame ID	overhead_camera/mount/overhead_camera	Validated
Image publishing rate	~0.79 Hz	Measured
CameraInfo publishing rate	~0.76 Hz	Measured
Timestamp progression	0.5 s increments in sampled sequences	Observed
Detection topic	PhoneDetectionArray publisher active	Validated

Detection publication rate	~0.80 Hz observed	Measured
Exact image/CameraInfo pair synchronisation	Not established by sampled evidence	Limitation
Controlled dropped-frame experiment	Not performed	Limitation

11. Discussion

The main outcome of CV-20 is that the existing camera-to-detector QoS configuration was shown to be internally compatible in the tested Gazebo environment. Both camera streams use RELIABLE delivery and VOLATILE durability at the publisher, while the perception node requests the same settings. No QoS reliability mismatch was observed.

The rate measurements also showed that the running simulation delivered camera messages at a relatively low rate. Because there was no controlled comparison between QoS profiles, it would be inappropriate to attribute that behaviour to QoS. From an engineering perspective, the more useful conclusion is that the measured runtime rate should be considered when interpreting perception latency and when the same pipeline is later connected to physical hardware.

The timestamp analysis reinforced the difference between stream consistency and exact synchronisation. Matching frame IDs and regular timestamp progression are positive indicators, but the evidence collected here did not contain overlapping image and CameraInfo timestamps. A future physical-camera validation should therefore include an explicit matched-message timestamp test if exact pairing is required.

12. Limitations and Future Validation

- The image and CameraInfo timestamp sequences were collected in separate sampling windows, so exact message-to-message synchronisation was not demonstrated.
- A deliberate dropped-frame or delayed-frame fault injection was not performed; therefore no quantitative packet-loss resilience claim is made.
- The measured camera rates reflect the active Gazebo/WSL runtime and should not be treated as the nominal physical-camera rate.
- The /phone_detections rate is affected by detector processing as well as camera input and cannot alone establish camera frame loss.
- The investigation focused on validating the existing configuration and did not require a QoS code/configuration change because no compatibility mismatch was observed.

For later physical-camera testing, the most useful follow-up would be to capture image and CameraInfo messages in the same time window and calculate timestamp differences between matched messages. A controlled interruption or delay test could then quantify recovery behaviour under dropped or delayed frames.

13. Personal Reflection

This contribution helped me understand that ROS 2 integration issues are not necessarily caused by the computer vision model itself. It would be easy to focus only on whether the detector produces a result, but CV-20 required me to examine how the camera data actually moves through the ROS 2 graph.

I found the QoS inspection particularly useful because it changed an assumption about communication into something that could be verified directly from the running system. The timestamp investigation was also a useful reminder to be precise with technical claims. The streams looked consistent, but because the timestamp samples were collected in different windows, I could not honestly describe them as exactly synchronised. I consider this an important part of engineering validation: the conclusion should match the evidence, even when the evidence is less complete than the ideal test.

14. Conclusion

CV-20 validated the camera communication and timing characteristics of the current ROS 2/Gazebo perception pipeline. The overhead image and CameraInfo publishers use RELIABLE and VOLATILE QoS, and the onnx_phone_detector subscribes to both using the same settings. No QoS compatibility problem was observed. Both streams use the same overhead-camera frame ID, while their observed publishing rates were approximately 0.79 Hz for images and 0.76 Hz for CameraInfo.

Timestamp sampling showed regular progression in 0.5-second simulation-time increments within each stream, and the custom /phone_detections interface was confirmed to be active and publishing at approximately 0.80 Hz during the test. The main limitation is that exact pairwise image-to-CameraInfo synchronisation was not established because the timestamp samples were collected in separate windows. Overall, the investigation provides a documented baseline for the current Gazebo integration and identifies the specific synchronisation evidence that should be added when the project moves towards physical-camera validation.